

Manifeste du développement assisté par l'IA agentique

Le craftsmanship à l'ère des agents



Livre blanc v2.0 — Juin 2026



Table des matières

L'essentiel	3
Chapitre 1 — Productivité à l'ère de l'IA	5
Chapitre 2 — Souveraineté numérique, risques et impacts	6
Chapitre 3 — L'organisation des équipes de développement	12
Chapitre 4 — La transformation des métiers	14
Chapitre 5 — La transformation de l'organisation et la conduite du changement	19
Chapitre 6 — Le cycle de vie du développement logiciel piloté par l'IA	25
Chapitre 7 — Le Specification Driven Development (SDD)	29
Chapitre 8 — L'AI IDP (Internal Developer Platform)	31
Chapitre 9 — Le Context Nexus : la documentation comme infrastructure agentique	34
Chapitre 10 — Gouvernance du PDLC et métriques	38
Conclusion — Checklist de maturité IA pour le CTO	41
Annexes	42

L'essentiel

L'IA générative accélère le développement logiciel — mais sans méthodologie adaptée, elle peut créer autant de problèmes qu'en résoudre. Ce livre blanc propose un cadre pour transformer cette accélération en performance organisationnelle durable.

Il s'inscrit dans le **framework AI Multiplier** de Zenika, qui couvre l'ensemble du périmètre opérationnel d'une transformation IA selon trois disciplines complémentaires — **Shape** (stratégie, *discovery*, design), **Ship** (ingénierie et delivery) et **Sync** (architectures et plateformes). Ce document a un focus opérationnel, plutôt tourné **Ship**, mais s'intéresse particulièrement aux **intersections** avec les autres disciplines — condition *sine qua non* au succès de la transformation dans la durée.

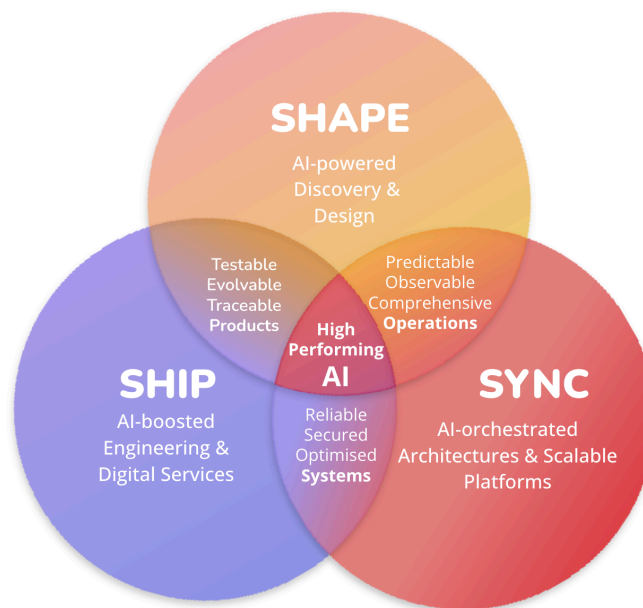


Figure 1. – Framework AI Multiplier : trois disciplines dont l'intersection produit l'effet multiplicateur.



L'essentiel à retenir

📌 **L'IA sans méthodologie accélère la dette technique.** Le *Specification Driven Development* transforme la productivité individuelle en performance organisationnelle durable.

🔧 **Ne dépendez de personne.** Une plateforme AI débrayable — gateway, registry, context factory — garantit la liberté de basculer de modèle, fournisseur ou cloud à tout moment.

❤️ **Craftsmanship (artisanat logiciel) + AI = ❤️.** Les **instructions de développement partagées** scalent la qualité au-delà de la discipline individuelle.

⚖️ **Mesurez la valeur, pas le volume.** L'IA se mesure aux frictions systémiques qu'elle élimine, pas au code qu'elle génère.

PARTIE I

L'enjeu



Chapitre 1 — Productivité à l'ère de l'IA

L'IA accélère, mais qui vérifie ?

1.1. Le paradoxe de la productivité

Le marché de l'IA générative appliquée au développement logiciel atteint un point de bascule. Les chiffres d'adoption sont spectaculaires : 72 % des développeurs utilisent l'IA quotidiennement (Sonar 2026) et 84 % utilisent ou prévoient d'utiliser des outils IA (Stack Overflow 2025).

Les gains de productivité individuels sont réels : le rapport DORA 2025 mesure 21 % de tâches supplémentaires accomplies par développeur. Toutefois, ce gain individuel ne se convertit pas mécaniquement en performance de livraison à l'échelle de l'équipe.

96 %

ne font pas
totalement
confiance à la
justesse de l'IA

Sonar 2026

42 %

du code committé
est généré par l'IA

Sonar 2026

29 %

niveau de
confiance envers
l'IA (-11 pts)

Stack Overflow 2025

27 %

revoient l'intégralité
du contenu IA
avant usage

McKinsey 2025

1.2. La dette invisible

Les équipes qui génèrent plus de code avec l'IA connaissent davantage de défaillances en production, pas moins. Seuls 48 % des développeurs vérifient systématiquement le code avant commit (Sonar 2026), et 45 % pointent comme frustration première les solutions « presque correctes mais pas tout à fait » (Stack Overflow 2025) — le code est écrit plus vite, mais les revues, les tests et le déploiement ne suivent pas proportionnellement. DORA conceptualise ce phénomène en 2026 sous le nom de **verification tax** : l'effort économisé sur le code boilerplate est remplacé par la charge cognitive nécessaire pour auditer un code qui *ressemble* à du code correct.

L'IA, entraînée sur du code existant contenant déjà des défauts, génère davantage de dette technique par ligne que les développeurs humains. Pour les organisations déjà endettées techniquement, c'est **20 à 40 % du budget IT** qui est absorbé par cette dette.



Notre conviction chez Zenika

Face à cette crise de la responsabilisation, la solution ne réside ni dans l'abandon de l'IA, ni dans une adoption aveugle de plus d'outils. Elle réside dans **une méthodologie standardisée, souveraine et vérifiable** : passer d'une utilisation opportuniste de l'IA à une **industrialisation méthodique du développement assisté**, portée par une méthodologie garantissant que les spécifications sont le nouveau pivot de tout développement à long terme (*Specification Driven Development*), et un framework technique débrayable.

Chapitre 2 — Souveraineté numérique, risques et impacts

Briser la dépendance aux géants du Cloud, limiter les risques économiques et organisationnels, limiter les impacts environnementaux.

Le paradoxe de productivité du chapitre 1 a un corollaire économique : **l'agentique coûte cher**, et son ROI dépend autant de la maîtrise de ce coût que de l'effort méthodologique.

2.1. Le piège du SaaS : analyse du TCO

L'adoption massive des assistants de code SaaS crée une dépendance structurelle dont peu d'organisations mesurent le coût réel. GitHub Copilot et Claude Code s'imposent comme des standards de facto, mais leur modèle économique repose sur un mécanisme de lock-in progressif — et **change de nature courant 2026** : la phase de subvention des modèles agentiques prend fin chez l'ensemble des éditeurs majeurs, au profit d'une facturation indexée sur la consommation réelle de tokens qui pénalise les usages intensifs.

Le coût total de possession (TCO) cumule cinq dimensions sur la durée : le **coût de licence** — désormais usage-based, donc directement proportionnel à l'intensité d'usage agentique de l'équipe ; le **coût d'infrastructure** (compute, GPU, stockage, opérations de maintenance, mises à jour — minimal en SaaS, structurant en souverain) ; le **coût de migration** (effort de sortie d'un fournisseur lorsque les tarifs ou conditions changent — proxy du *vendor lock-in*) ; la **protection de la propriété intellectuelle (PI)** (effort à provisionner pour empêcher qu'une partie du code source ou de la documentation transitant par des serveurs tiers soit exfiltrée, journalisée ou réutilisée pour l'entraînement) ; et le **risque méthodologique** (dont le tuning) — un coût présent dans tous les scénarios (instructions partagées, calibration des prompts, vérification déterministe), qui croît fortement dans le scénario souverain pour combler l'écart de qualité résiduel des modèles Open Weight et la moindre maturité opérationnelle des stacks d'auto-hébergement.

L'émergence rapide des modèles chinois (Qwen3 Coder, GLM-4.6, DeepSeek, MiniMax) — désormais comparables aux meilleurs modèles propriétaires US/UE sur la génération de code — introduit un quatrième scénario : **Full SaaS via API chinoises**. Le coût brut des tokens y est divisé par environ dix, mais le risque IP / réglementaire explose : cadre légal chinois (Data Security Law, PIPL, National Intelligence Law) qui prévoit l'accès des autorités, absence de mécanisme de transfert de données équivalent au RGPD, opacité sur la rétention des prompts. Un risque existe certes aussi avec les acteurs américains, mais sa nature diffère : en cas de contentieux, un contrat reste généralement exécutable face à un fournisseur US, là où il ne l'est qu'exceptionnellement face à un acteur chinois. Pour des organisations européennes — *a fortiori* sur secteurs régulés — l'arbitrage économique pur favorise cette option, mais le surcoût de couverture du risque l'efface en grande partie. La vraie option de valeur se situe ailleurs : exécuter ces mêmes pondérations open-weight **en auto-hébergé**, ce qui rejoint le scénario Souverain.

Cette option auto-hébergée demande toutefois un point de veille spécifique. La stratégie industrielle chinoise début 2026 vise explicitement à se découpler de l'écosystème Nvidia : la *Cyberspace Administration of China* a interdit aux principaux acteurs locaux d'acquérir du matériel Nvidia, quand les Etats-Unis interdisaient également l'export de ces cartes sur le marché Chinois. Conséquence difficile à anticiper, pour des raisons géopolitiques qui échappent aujourd'hui à l'ensemble des acteurs : la compatibilité des modèles chinois open-weight avec le matériel Nvidia

pourrait se dégrader à un horizon de 12 à 24 mois, à mesure que kernels, opérateurs et recettes de quantisation sont finement ajustés pour les puces Ascend de Huawei. Pour éviter d'être pris en étau entre le monopole de CUDA en Occident et la montée en puissance de CANN en Chine, il sera crucial de s'appuyer sur les travaux d'acteurs comme Red Hat, dont l'objectif est justement de développer des frameworks d'inférence agnostiques capables de faire abstraction de ces deux environnements propriétaires.

Cette dépendance aux éditeurs étrangers pourrait se justifier si elle se traduisait par des retours sur investissement à la hauteur. Or, McKinsey observe que seulement 21 % des organisations ont véritablement redessiné leurs workflows pour intégrer l'IA. Les 79 % restants superposent l'IA à leurs processus existants, maximisant ainsi leur exposition au risque de dépendance sans en tirer les bénéfices organisationnels.

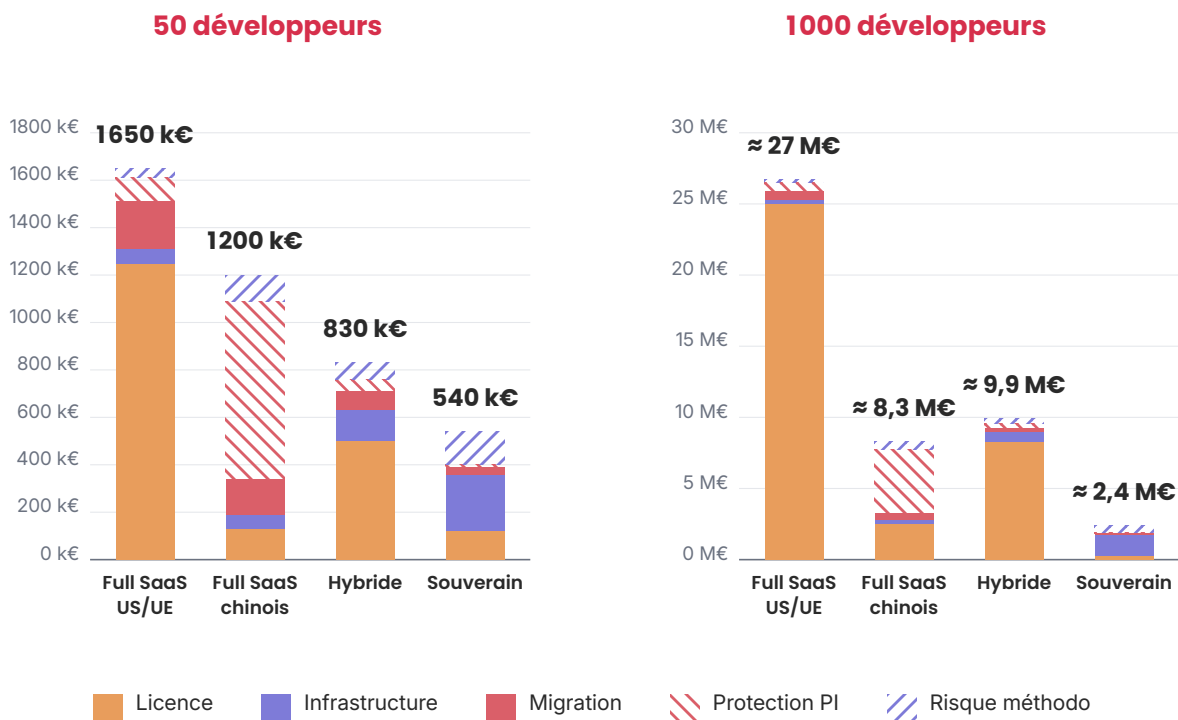


Figure 2. – TCO sur 3 ans selon la taille d'équipe, après la bascule usage-based intervenue courant 2026. À gauche : 50 développeurs (PME / mid-market). À droite : 1000 développeurs (grand compte). Hypothèses : moyenne pondérée ≈ 600 € / dev / mois pour les modèles US/UE, ≈ 60 € pour les modèles chinois en API ; croissance +15 %/an de la consommation ; coûts fixes (méthodo, infra) amortis sur l'effectif. Ceci exclut les usages autres que la génération de code.

2.2. Des impacts environnementaux qui augmentent avec les coûts

L'IA générative repose sur de grands modèles de langage qui sont construits à partir de gros réseaux de neurones logiciels, avec des milliards de paramètres.

Pour les faire fonctionner, les exploitants utilisent des équipements énergivores. Afin de suivre l'augmentation en puissance de ces équipements, les datacenters doivent également évoluer : leur consommation d'électricité augmente en ordre de grandeur pour arriver à 1 GW (plus que la production d'un réacteur nucléaire, 0,8 GW) en France, voire 5 GW pour Hypérion de Meta et 10 GW pour Stargate d'OpenAI.



Cette demande en électricité entraîne une augmentation des prix, mais aussi une hausse des émissions de gaz à effet de serre à travers le retardement de la fermeture de centrales à charbon, ou encore l'utilisation de turbines à gaz.

L'ampleur de ces émissions dépend toutefois fortement du mix électrique du lieu de calcul : un même traitement émet bien davantage sur un réseau charbonné (Chine, environ 560 gCO₂/kWh ; États-Unis, environ 370 g) que sur un réseau bas carbone comme celui de la France (quelques dizaines de gCO₂/kWh, grâce au nucléaire), pour une moyenne mondiale autour de 445 g (Ember, *Global Electricity Review* 2025). À volume égal, le coût écologique d'un token n'est donc pas le même selon l'endroit où il est exécuté : un argument supplémentaire en faveur d'une exécution souveraine et localisée.

Ces constatations, associées à la progression exponentielle des commandes, amènent à des projections de croissance des impacts du numérique très impressionnantes, surtout dans un contexte où ceux-ci devraient décroître (comme pour tous les secteurs).

Les projections de l'Agence Internationale de l'Énergie sont de 945 TWh consommés par les datacenters dans le monde en 2030, contre 415 en 2024. Le Shift Project, plus pessimiste, table sur 1500 TWh et 920 MtCO₂e/an (soit deux fois la France). Dans les deux cas, en majorité du fait du développement de l'IA générative.

Cependant, il est possible que les limitations en disponibilité des matériaux (métaux), composants (RAM, batteries, onduleurs, etc.) et capitaux, couplées à l'optimisation des modèles, limitent cette croissance des impacts négatifs.

À l'inverse, un effet rebond (paradoxe de Jevons) peut jouer en sens contraire : à mesure que le coût par token baisse, les usages se multiplient, si bien que les gains d'efficacité unitaires ne garantissent pas une diminution de l'impact total.

2.3. Risque économique et organisationnel

L'utilisation des produits d'IA générative entraîne, comme pour tout achat de service, une dépendance aux fournisseurs. Or leur modèle économique est encore fragile : ces acteurs sont pour la plupart dépendants d'apports de capitaux. Le risque est que certains d'entre eux disparaissent dans les années qui viennent ou que leurs produits deviennent excessivement chers, comme semblent l'indiquer les récentes évolutions des tarifs et la limite parfois floue du nombre de tokens par licence.

L'utilisation de modèles de fournisseurs différents, open weight ou locaux, est toujours possible mais nécessite une réorganisation des équipes et des outils.

Ce risque est d'autant plus à prendre en compte que les modifications des habitudes et compétences des collaborateurs induites par l'utilisation des outils d'IA générative rendent un retour arrière difficile.

2.4. L'alternative Open Weight : une nécessité économique et réglementaire

La souveraineté technologique n'est plus une question éthique ou philosophique. C'est une nécessité économique et réglementaire.

2.4.1 La nécessité économique

Les modèles Open Weight progressent vite et offrent des performances compétitives pour la génération de code en entreprise : Mistral Small 4 surpasse des modèles 120B sur LiveCodeBench tout en produisant 20 % moins de tokens, et DeepSeek Coder V2, Qwen 2.5/3.5 et Llama 3.3 sont solides. Ces modèles sont exécutables localement avec 30 à 60 Go de RAM, limitent la dépendance à une API tierce et les coûts récurrents associés.

Reste à reconnaître deux nuances. D'abord, à coût d'exécution raisonnable (un poste local, un serveur GPU départemental), **les meilleurs modèles propriétaires gardent une légère longueur d'avance** sur les tâches les plus exigeantes — le scénario souverain doit donc budgéter un effort supplémentaire de tuning, d'embeddings spécialisés, de calibration des prompts et de méthodologie SDD renforcée. Ensuite, **la stack souveraine est récente** :

auto-hébergement de modèles, AI Gateway, vector stores et MLOps internes ne disposent pas du même recul opérationnel que les solutions SaaS établies — la pénurie de talents est réelle et le risque d'obsolescence matérielle ou éditeur (rachat, pivot stratégique d'un fournisseur Open Weight) ne peut être ignoré.

Limite à connaître

Les modèles Open Weight les plus puissants ne tournent pas encore sur des postes de travail standard : ils exigent une **infrastructure dédiée** (GPU, mémoire, réseau). Le terrain bouge vite — une **veille continue** est cruciale pour ajuster les seuils de bascule local/cloud et adapter les bonnes pratiques au fil des évolutions matérielles et logicielles.

2.4.2 La nécessité réglementaire



Calendrier de l'AI Act européen

- **Févr. 2025** : interdictions des pratiques IA à risque inacceptable — *en vigueur*.
- **Août 2025** : obligations pour les modèles GPAI — *en vigueur*.
- **Août 2026** : application des règles *high-risk* initialement prévue — **report en cours** via le paquet *Digital Omnibus*.
- **Déc. 2027** (proposition de la Commission) : application des règles *high-risk* stand-alone.
- **Août 2028** (proposition de la Commission) : application aux systèmes *high-risk* embarqués dans des produits régulés.

Les négociations entre le Parlement, le Conseil et la Commission du 29 avril 2026 ayant échoué, le calendrier reste à confirmer ; une nouvelle session de négociation est attendue dans les semaines à venir.

Comme le souligne Mistral dans son livre blanc sur la souveraineté IA en Europe, les nations doivent construire leurs propres écosystèmes IA pour préserver leur autonomie numérique. Cette logique s'applique à l'échelle de l'entreprise : disposer de la capacité à exécuter des modèles en local, c'est garantir sa liberté de choix future.

35 % des développeurs accèdent aux outils IA via des comptes personnels plutôt que des comptes sanctionnés par l'entreprise, créant un angle mort pour la sécurité et la conformité (Sonar 2026). Le traitement local du code source répond directement à cette préoccupation et simplifie la conformité RGPD, indépendamment des évolutions de l'AI Act.

2.5. L'approche débrayable Zenika

Notre conviction est claire : **il ne s'agit pas de rejeter les solutions des grands fournisseurs, mais de ne jamais en dépendre.**

L'approche que nous préconisons est celle du « best of breed débrayable ». Elle repose sur trois principes :



Abstraction

Une couche d'**AI Internal Developer Platform** (AI IDP) découple le workflow de développement du provider de modèle. Basculer d'un modèle propriétaire à un modèle Open Weight, d'une API cloud à une exécution locale, se fait par configuration.



Outillage agnostique

Le framework repose sur des outils open-source et interopérables plutôt que sur des IDE propriétaires verrouillés. Chaque brique est remplaçable.



Plan B permanent

Chaque organisation peut, à tout moment, migrer son workflow vers des modèles locaux sans perte de fonctionnalité. Une capacité testée et maintenue.

Notre expérience terrain confirme cette posture. Zenika accompagne de nombreuses organisations qui maîtrisent déjà leur propre infrastructure pour des raisons de souveraineté (santé, finance, secteur public...). Dans tous les cas, le choix se porte sur l'**approche hybride** assortie d'une **politique de gestion des données claire et contrôlée** — pas sur le tout-souverain. Le constat est partagé : la valeur ne vient pas de la pureté du scénario mais de la maîtrise du flux de données et de la capacité à débrayer.

Comme le rappelle MIT Technology Review en 2026, la **souveraineté pure est un mythe** — la chaîne d'approvisionnement matérielle reste mondiale. Mais cette nuance ne disqualifie pas l'effort : la souveraineté **graduelle**, débrayable et pilotée, protège contre les hausses de tarifs unilatérales, les changements de conditions d'utilisation, les décisions géopolitiques affectant l'accès aux services cloud, et les exigences réglementaires futures. Elle transforme un compromis difficile en avantage concurrentiel.

PARTIE II

Transformation & organisation



Chapitre 3 — L'organisation des équipes de développement

Où se situe votre organisation, et quelles frontières l'IA redessine-t-elle ?

Pour dompter l'agentique et la transformer en levier de ROI, il faut d'abord savoir d'où l'on part. La matrice qui suit positionne chaque organisation sur ce chemin.

3.1. Matrice de maturité IA : où en est votre organisation ?

Pour aider les décideurs à se positionner, nous proposons une grille d'auto-diagnostic en quatre niveaux de maturité — *Explorateur*, *Structurateur*, *Facilitateur*, *Propulseur* —, présentés ci-dessous du plus avancé (haut) au plus initial (bas), avec un radar à cinq axes pour visualiser son profil. L'objectif est de situer l'organisation dans ses pratiques de développement assisté par IA, de l'usage individuel à la mise en place d'outils d'entreprise, méthodologies et plateforme interne de développement favorisant l'IA générative (AI IDP), afin de dessiner une trajectoire visant à optimiser ces pratiques.

NIVEAUX DE MATURITÉ

4

Propulseur

Le développement assisté est pleinement intégré à la culture d'ingénierie. Des chaînes multi-agents (orchestrator + sous-agents spécialisés) sont en production, et l'AI IDP (gateway, registry, context factory) assure l'indépendance fournisseur. L'équipe est référente IA dans l'écosystème.

3

Facilitateur

Un workflow structuré encadre la production : la spécification précède la génération, la validation est systématique (tests, analyse statique, revue), et les conventions sont versionnées via des instructions de développement partagées. La souveraineté est identifiée comme enjeu.

2

Structurateur

L'organisation a déployé un ou plusieurs assistants de code (Copilot, Cursor) sur des tâches ciblées. La vérification existe mais reste partielle : les gains de productivité individuelle masquent une dette technique croissante.

1

Explorateur

L'IA est utilisée individuellement par quelques développeurs curieux, sans cadre ni gouvernance. Les outils sont choisis au gré des préférences personnelles, et le savoir ne se diffuse pas à l'équipe.

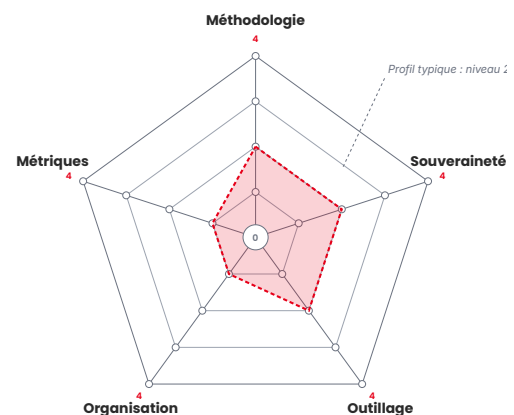


Figure 3. – Radar de maturité IA et description des quatre niveaux. Le profil illustré (en pointillé rouge) correspond à une organisation typique au stade « Structurateur ».



Prévision Gartner

40 % des applications d'entreprise intégreront des agents IA spécifiques d'ici fin 2026, contre moins de 5 % en 2025.

Mais la même étude avertit que **40 %** des projets d'IA agentique seront abandonnés d'ici fin 2027 — précisément parce qu'ils manquent de cadre méthodologique.

L'écart entre le **niveau 2 — Structurateur** (où se situe la majorité des organisations qui ont déployé Copilot, Cursor ou équivalents sans cadre méthodologique) et le **niveau 4 — Propulseur** (où le framework et l'AI IDP sont opérationnels) est précisément la zone à risque que pointe Gartner. Sans méthodologie ni indépendance fournisseur, les pilotes ne franchissent jamais le passage à l'échelle.

3.2. La fusion des rôles autour de la spécification

La loi de Conway stipule que les systèmes logiciels reflètent la structure de communication de l'organisation qui les produit. À l'ère du développement assisté par IA, cette loi prend une dimension nouvelle : si la méthodologie change, l'organisation doit suivre.

L'artefact de spécification devient le point de convergence de ces trois rôles. C'est autour de lui que s'organisent les revues, les validations et les itérations — remplaçant les silos traditionnels par un flux continu.

McKinsey révèle que 51 % des entreprises ont déjà constaté des conséquences négatives de l'adoption de l'IA, et seulement 33 % réussissent à la déployer à l'échelle organisationnelle. Les deux tiers restent bloqués en «pilote permanent». La cause n'est pas technique — elle est organisationnelle. Tant que les rôles ne se reconfigurent pas autour de la spécification, le passage à l'échelle reste hors de portée.

Le *Specification Driven Development* redéfinit les frontières entre les rôles traditionnels.



Le Développeur

N'est plus un rédacteur de code. Il devient un *éditeur de spécifications* qui définit les intentions, valide les artefacts générés et maintient la cohérence du système.



L'Architecte

Son rôle s'étoffe : au-delà de la conception technique, il devient un *ingénieur de contexte* dont la valeur réside dans sa capacité à transformer les connaissances en contexte actionnable pour les agents.



Le Product Owner

Gagne en autonomie : il est responsable de l'intention métier, exprimée dans un langage de spécification clair qui réduit la friction avec l'équipe technique.

CHAPITRE CO-ÉCRIT AVEC WAVESTONE



Chapitre 4 — La transformation des métiers

Piloter la recomposition des métiers, des compétences et des trajectoires.

L'IA appliquée au développement logiciel a changé d'échelle : son impact se lit désormais dans le compte de résultat global de l'entreprise, au-delà du seul budget technique. 88 % des entreprises observent déjà un effet de l'IA sur leur chiffre d'affaires annuel, dont 30 % le jugent significatif (NVIDIA, *State of AI 2026*) ; cet effet porte sur les revenus comme sur les marges.

Un impact de cette ampleur engage l'entreprise entière : il dépasse le périmètre de la DSI et devient un enjeu d'organisation. Quand l'IA recompose le travail, elle recompose les rôles individuels comme les structures d'équipe, et le CTO qui pilote ses outils sans piloter cette recomposition perd le contrôle technique en même temps que les personnes nécessaires pour le maintenir.

Cette recomposition se joue à deux échelles, que ce chapitre et le suivant traitent successivement : celle des **métiers, des compétences et des trajectoires individuelles** d'abord ; celle des **structures d'équipe et de la conduite du changement** ensuite (chapitre 5).

Zenika x Wavestone : pourquoi cette alliance

Ces deux chapitres portent sur la transformation des métiers et de l'organisation : un terrain sur lequel Zenika et Wavestone interviennent conjointement dans le cadre de programmes de transformation à grande échelle.

En effet, le principal frein n'est pas tant l'adoption des outils que la transformation du PDLC (Product Development LifeCycle) dans son ensemble : modes de collaboration, pratiques d'ingénierie, gouvernance et organisation. C'est précisément là que la complémentarité entre Wavestone et Zenika prend tout son sens.

Wavestone structure la transformation en alignant les parties prenantes, en définissant la gouvernance et l'organisation cible, et en pilotant la conduite du changement pour garantir une adoption durable et mesurable. **Zenika**, de son côté, ancre cette transformation dans le réel grâce à l'excellence du delivery produit *AI-native*, des pratiques d'ingénierie éprouvées et une forte proximité avec l'opérationnel.

4.1. De l'expérimentation à l'urgence : la trajectoire 2025-2026

2025 a été l'année de l'expérimentation diffuse : les équipes ont testé leurs outils en autonomie, sans pilotage central. La phase était légitime, il fallait laisser émerger les usages. Et la courbe d'adoption n'a jamais été aussi rapide dans l'histoire du logiciel : 90 % des équipes d'ingénierie déclarent un usage de l'IA dans leurs flux de travail à fin 2025, contre 61 % un an plus tôt (Exceeds AI, 2026).

Au premier trimestre 2026, le réflexe naturel s'est déclenché : un plan de formation, avec catalogue de modules, cohortes et certifications. Les initiatives se sont multipliées, mais les taux de complétion stagnent et l'effet sur la livraison ne se matérialise pas. McKinsey le confirme (*State of AI 2025*) : seulement 39 % des organisations parviennent à relier un impact EBIT à l'IA, et plus de 80 % ne constatent encore aucun effet sur leur résultat global. Quelque chose résiste, et ce n'est pas l'adoption.

Au second trimestre 2026, le diagnostic se précise, et il remonte désormais des responsables informatiques eux-mêmes. Michael Morris (Randstad Digital, 2026) le résume : « l'IA d'entreprise n'échoue pas au niveau du modèle ; elle échoue à la couche d'implémentation. Augmenter la vitesse des outils sans augmenter la capacité des ingénieurs à les gouverner produit de la dette technique à l'échelle. » Le sujet a changé de nature : il porte désormais sur le cycle de production lui-même, et sur la composition des équipes qui le font tourner.



Quand le coût de l'expérimentation retombe sur les individus

Pendant la phase d'expérimentation 2025, beaucoup d'organisations ont laissé leurs équipes essayer librement les outils IA, sans cadre ni grille d'observation. La promesse implicite était : « débrouillez-vous, on verra bien ce qui marche ».

Cette posture a fait reposer le coût de l'expérimentation sur les individus, non sur l'organisation : du temps personnel investi, des échecs vécus comme des défaites individuelles faute d'être partagés. À l'entrée de 2026, les profils les plus engagés ne savent plus quoi montrer de leur expérimentation, et tous découvrent que ce qu'ils ont appris n'est ni capitalisé ni valorisé dans leur évaluation.

L'expérimentation ne produit de la valeur que si l'organisation se donne les moyens d'en lire les résultats.

4.2. Une transformation plutôt qu'une montée en compétence

Le réflexe formation repose sur un postulat implicite : un métier stable auquel on viendrait greffer une nouvelle compétence. Or le métier de développement ne se voit pas ajouter une compétence, il se recompose. La vraie question porte sur ce que devient le travail et sur qui le réalise. Tant que le diagnostic reste au niveau de la compétence individuelle, il manque l'essentiel. Trois réflexes l'y enferment (Figure 4).



Figure 4. – Les trois réflexes qui enferment le diagnostic au niveau de la compétence individuelle, alors que le métier se recompose.

Le premier est de **répondre par l'outil**. Vu depuis la seule DSI, la transformation IA devient un problème d'outillage : choisir les outils, sécuriser les flux, former à la rédaction de prompts. Tout cela est nécessaire, mais la réponse par l'outil ne résout jamais un problème de transformation. On équipe des rôles qui ne ressemblent déjà plus à ce qu'ils étaient, et on confond l'équipement avec l'accompagnement.

Le deuxième est d'**évaluer comme avant**. Le métier se recompose, mais les critères de recrutement et les grilles d'évaluation restent calés sur l'ancien. Tant qu'on note principalement la qualité du code écrit à la main plutôt que la capacité à orchestrer, vérifier et intégrer une production assistée, on continue à survaloriser des compétences dont le poids relatif diminue dans le métier. Les fondamentaux du développement restent nécessaires, mais ils ne suffisent plus à caractériser la performance des profils les plus efficaces dans un environnement assisté par l'IA.

Le troisième est de **croire que seuls les développeurs sont concernés**. Le PDLC concerne toute la chaîne de production logiciel : produit, design, QA, ops. Quand la spécification redevient le contrat exécutable du système (cf. chap. 7 sur le SDD), sa revue et parfois sa rédaction reviennent aux expertes et experts produit autant qu'aux profils techniques. Dans certaines organisations, les plus matures, des chefs de produit livrent désormais des fonctionnalités en production via Cursor, Claude Code ou v0, sous revue d'ingénieurs mais sans dépendre de leur file d'attente. La production de valeur a toujours dépassé la simple production de code, et ce paradigme est d'autant plus vrai aujourd'hui. Ce glissement transversal ouvre des opportunités réelles, et déstabilise les organisations qui ne l'ont pas anticipé.

Et les profils techniques le voient venir. Selon le Dev Barometer Q4 2025 de BairesDev (501 répondants, en majorité 8 ans et plus d'expérience), 65 % des seniors estiment que leur rôle sera redéfini par l'IA en 2026 : 74 % anticipent un glissement du codage vers la conception de solutions, et 58 % prévoient une réduction des tâches d'entrée de gamme.

Le défi se situe côté management : prendre la mesure d'une mutation longtemps difficile à anticiper, puis accompagner des personnes dont le rôle change, plutôt que se limiter à promouvoir l'adoption des outils.

Il convient toutefois de distinguer transformation du métier et disparition du métier. Les fondamentaux de l'ingénierie logicielle (conception, architecture, débogage, maîtrise de la complexité, compréhension des systèmes) demeurent. Ce déplacement est comparable à celui qu'ont connu les administrateurs systèmes avec l'automatisation : la connaissance profonde du système reste indispensable, même lorsque l'exécution directe devient plus rare. L'enjeu n'est pas leur rempla-

cement, mais leur déplacement vers des activités où le jugement humain conserve un avantage décisif.

4.3. La recomposition des trajectoires individuelles

L'IA recompose le métier de développement, et elle touche les deux extrémités de l'échelle de carrière à la fois : les seniors comme les débutants.

Les développeurs les plus aguerris sont souvent des artisans du code (*craftspeople*) : ils ont choisi ce métier par goût du code, et leur identité s'est construite sur le geste technique et le plaisir d'un système bien conçu. Or l'IA absorbe précisément les tâches où se forgeait cette identité. Auditionné le 12 mai 2026 par la commission d'enquête de l'Assemblée nationale sur les vulnérabilités numériques, Arthur Mensch (CEO de Mistral AI) le résume sans détour : « les ingénieurs chez Mistral n'écrivent plus de lignes de code. Vous n'êtes plus un artisan, vous êtes un manager. » Cette situation n'est pas nécessairement représentative de l'ensemble de l'industrie, mais elle illustre une direction que certaines organisations très avancées explorent déjà. Pour ces profils, le passage de l'artisan au manager se vit souvent comme une perte : celle du geste qui les attachait au métier. L'inquiétude redouble quand arrivent des juniors qui orchestrent l'IA avec plus d'aisance qu'eux : le risque de se sentir dépassé est réel.

Cette inquiétude est légitime, et elle a une contrepartie précieuse. Le regard critique de ces développeurs sur le code, leur capacité à repérer ce qui cloche dans une sortie qui « semble bonne », est exactement ce dont l'organisation a besoin face à l'IA. Sans eux, l'entreprise glisse vers du *vibe engineering* : on accepte ce que produit l'agent parce que ça compile et que les tests passent, sans plus interroger la pertinence ni la maintenabilité.

Au-delà de la seule productivité

Mesurer l'IA à la seule vélocité, c'est passer à côté de l'essentiel. Son apport le plus durable est ailleurs : une **exigence de qualité** réoutillée (revue, tests, vérification déterministe) et une **capacité de travail libérée** pour traiter ce qu'on remettait toujours à plus tard. Dette technique, documentation, tests de non-régression, accessibilité, observabilité : autant de chantiers que « personne n'avait le temps de faire », ou dont « personne ne voulait », et qui redeviennent abordables. La bonne question n'est pas « combien de lignes en plus ? » mais « quelles fragilités systémiques pouvons-nous enfin résorber ? ».

À l'autre bout de l'échelle, le canal qui alimentait la juniorité est lui aussi fragilisé. Les tâches routinières qui formaient les débutants, du code répétitif aux écrans simples, sont les premières automatisées, et ce sont celles que la plupart des cursus, écoles comme formations intensives de reconversion, apprenaient à produire. On ne peut plus former les juniors en leur transmettant les bases que maîtrisaient leurs aînés ; il faut les imprégner de qualité logicielle dans un monde où le code n'est plus écrit à la main. Le déplacement de la barrière d'entrée, de la syntaxe vers la compréhension d'un domaine, ouvre d'ailleurs le métier à des profils à forte expertise métier : juristes, médecins, logisticiens, etc. Leur connaissance devient

directement exploitable dans le cycle de développement.

Reconstruire ce canal d'apprentissage suppose d'outiller la transmission autant que de la préserver. D'un côté, intégrer dans nos processus des **agents IA d'accompagnement**, nourris du contexte réel de l'entreprise (base de code, conventions, domaine métier), pour guider les premières contributions des juniors sur des cas concrets plutôt que sur des exercices théoriques. De l'autre, raviver le **binôme et le pair programming**, seniors et juniors face au même problème, pour ne pas perdre, sous prétexte d'automatisation, la capacité d'accompagnement humain qui

forge le jugement. L'agent accélère la mise en autonomie ; le binôme transmet ce qu'aucun agent ne porte : l'exigence et le regard critique.

À cela s'ajoute un effet de volume : le recrutement junior recule déjà. Le Stanford AI Index 2026 mesure depuis 2022 une baisse de 20 % de l'emploi des développeurs et développeuses de 22 à 25 ans. Comme il faut des années pour former un senior, ce creux prépare une pénurie de seniors demain, ce que Matt Garman (CEO d'AWS) qualifie de « l'une des choses les plus stupides » que l'industrie puisse faire.

Opposer les générations serait une erreur ; l'enjeu est de rapprocher leurs pratiques. Les seniors détiennent l'expertise pour définir l'artisanat logiciel adapté à l'IA et en borner les bonnes pratiques : c'est désormais cela qu'ils transmettent, davantage que la syntaxe. Les juniors, eux, arrivent avec moins de préjugés et plus de virtuosité dans l'usage des agents. Faire grandir les deux ensemble, en nourrissant mutuellement leurs pratiques, recompose les trajectoires de carrière et dessine la manière de bien faire du logiciel de demain.

CHAPITRE CO-ÉCRIT AVEC WAVESTONE



WAVESTONE

Chapitre 5 — La transformation de l'organisation et la conduite du changement

Des métiers aux structures : recomposer l'organisation et embarquer les équipes, sans laisser le changement reposer sur les seuls volontaires.

Le chapitre précédent a montré comment l'IA recompose les métiers et les trajectoires individuelles. Cette recomposition ne s'arrête pas aux personnes : elle remonte aux **structures d'équipe**, aux **grilles d'évaluation** et à la **manière même de conduire le changement**. Piloter cette transformation demande autant de méthode que de doigté : partir des irritants réels plutôt que de la technologie, et faire des sceptiques des alliés plutôt que de les contourner.

5.1. La rupture organisationnelle

L'IA agentique resserre les cycles de delivery, et avec eux les périmètres d'équipe et les effectifs. La contraction se mesure déjà : sur plus de 300 organisations, les gains d'efficacité d'ingénierie convergent autour de 3 à 12 % (DX, Core 4 2026), bien en deçà des 21 % de tâches supplémentaires produites individuellement avec l'IA (chapitre 1). C'est le « paradoxe de la productivité » que documente DORA : le gain individuel ne se convertit que partiellement en performance d'équipe. Cette capacité libérée se reporte sur la cadence et la composition des équipes : on livre le même périmètre avec des cycles plus courts et des équipes plus réduites. À ce stade, le gain reste largement contrebalancé par la nécessité de mieux contrôler la production assistée (revue, tests, vérification), donc d'investir : une part de l'économie réalisée sur l'écriture du code est réaffectée à sa validation. Le recul de 20 % de l'emploi des développeurs et développeuses de 22 à 25 ans, relevé au chapitre 4, en est le symptôme le plus visible.

Cette pression s'exerce sur toutes les organisations, qu'elles soient encore silotées ou déjà passées au Build & Run. Les secondes sont mieux préparées, développement et exploitation y étant déjà découplés et les tests déjà en place, mais l'accélération du delivery les met elles aussi sous pression. Partout, l'IA redéfinit les rôles : un développeur orchestre de l'infrastructure via des agents, un profil ops supervise des agents plutôt que des scripts, et les grilles d'évaluation ne décrivent plus le travail réel.

Il faut donc repenser la structure plus en profondeur. D'abord, faire évoluer la façon dont on évalue les profils : noter un profil full-stack sur sa seule maîtrise des frameworks revient à le juger sur une compétence qui ne décrit plus son métier. Alan l'a montré en octobre 2025. Son épreuve de recrutement de 2013, vingt lignes de pseudo-code, était résolue en un seul prompt par GPT-3 ; en la conservant, l'entreprise « sélectionnait activement contre les ingénieurs qui avaient adopté les pratiques modernes ». Sa réponse : des phases sans IA pour le raisonnement d'architecture et de sécurité, des phases avec IA pour la qualité de la rédaction de prompts et le regard critique sur les sorties. Cette évaluation rénovée doit aussi confirmer et encourager les spécialistes : la sécurité,

la fiabilité, la conformité ne disparaissent pas parce qu'on a fusionné des rôles, et les profils qui les portent restent des garde-fous indispensables.

Ensuite, faire pivoter les équipes : davantage de compréhension métier, moins de code écrit à la main, dans des structures resserrées et autonomes sur le modèle de la *two pizza team*, responsable d'un périmètre de bout en bout. Encore faut-il piloter cette compression : réduite sans méthode ni accompagnement, une petite équipe perd en qualité ce qu'elle gagne en vitesse.

Repenser ainsi l'organigramme de compétences ramène au premier plan une responsabilité propre du CTO : composer ses équipes avec la bonne diversité de profils, orchestrateurs et garants de la qualité, seniors et juniors. La loi de Conway, rappelée au chapitre 3, en donne la raison : les systèmes logiciels reflètent la structure de communication de l'organisation qui les produit. Cette recomposition ne peut donc se déléguer entièrement à la direction RH ni se traiter comme un pur sujet technique : elle se pilote à deux, CTO et direction RH, selon une *Inverse Conway Maneuver*, en concevant d'abord l'architecture cible puis en dérivant l'organisation qui la produira.

Cette recomposition organisationnelle ne concerne pas uniquement la production. À mesure que des profils non traditionnellement techniques participent davantage à la création logicielle, les questions de responsabilité, d'auditabilité, de conformité et de sécurité prennent une importance croissante. *Plus la capacité de produire se diffuse, plus les mécanismes de contrôle doivent être explicites.*

5.2. Partir des irritants : la Discovery comme fondation

Devant cette rupture, le réflexe le plus coûteux est de commencer par la technologie : choisir une plateforme, déployer un agent, puis espérer l'adoption. Or la cause d'échec la plus répandue n'est pas technologique. Elle se joue avant tout choix d'outil, dans la confusion sur le problème que l'IA est censée résoudre. Le chapitre 2 l'a chiffré : seules 21 % des organisations ont véritablement repensé leurs flux de travail pour intégrer l'IA, les 79 % restantes se contentant de la superposer à l'existant (McKinsey). Ce sont précisément les premières qui en tirent une valeur réelle, en posant le problème métier d'abord et la technologie ensuite. À l'inverse, la majorité des initiatives progressent en largeur (multiplication des pilotes) mais pas en profondeur, faute d'une vision claire de la valeur pour leur propre activité (Eficode, *State of AI in Software Organizations 2026*, plus de 270 organisations).

C'est la logique de la phase de **Discovery** que nous plaçons en amont de toute transformation du cycle de production. Plutôt que de déployer un outil et d'attendre l'adhésion, on commence par identifier les **points de douleur** réels du cycle : entretiens terrain, ateliers collaboratifs à grande échelle, cartographie des workflows et des pratiques de delivery, analyse des irritants et repérage des opportunités à plus forte valeur. Cette lecture partagée débouche sur une **gouvernance collective, documentée et axée sur les besoins réels des équipes**, et sur une trajectoire d'adoption priorisée par la valeur (les *quick wins* d'abord) plutôt que par la hype.

Cette démarche fait basculer l'organisation d'un mode **réactif** (les initiatives individuelles non structurées et le « débrouillez-vous » du chapitre 4) vers une **stratégie documentée**, adossée à un diagnostic de maturité (cf. la matrice du chapitre 3). Elle objective ce qui, jusque-là, reposait sur l'intuition et l'effort individuel, et donne à la transformation un point de départ partagé plutôt qu'imposé.

Retour d'expérience client : une adoption centrée sur les usages

Dans un grand groupe du secteur de l'énergie que nous accompagnons dans sa transformation, l'adoption des outils IA au sein des équipes de développement restait faible. Le choix s'est porté sur une démarche pragmatique centrée sur les usages plutôt que sur l'outillage. Le travail a commencé par une *Discovery* à grande échelle : entretiens terrain et ateliers collaboratifs, cartographie détaillée des cycles et pratiques de développement, analyse des irritants et identification des cas d'usage IA prioritaires, *gap analysis*, puis construction d'une trajectoire d'adoption réaliste et co-construite avec les équipes.

Comprendre les freins avant de prescrire des solutions, impliquer les équipes dans la définition de la cible : c'est cette inversion, les besoins d'abord et la technologie ensuite, qui transforme une adoption subie en transformation appropriée.

5.3. Faire des sceptiques des validateurs critiques

La conduite du changement bute presque toujours sur les mêmes profils : les **réfractaires**. Le réflexe classique consiste à les contourner, en déployant auprès des volontaires et en attendant que les autres suivent. Notre conviction est inverse : il faut **intégrer les profils réfractaires au cœur du dispositif, non comme des freins, mais comme des validateurs critiques**. Dirigé vers la robustesse des solutions, leur scepticisme devient une ressource plutôt qu'un obstacle.

Ce scepticisme est d'ailleurs fondé, et le chapitre 1 l'a chiffré : défiance massive envers la justesse de l'IA, code qui « semble correct » sans l'être, et seulement 48 % des développeurs qui vérifient systématiquement avant de valider (Sonar 2026). Le regard critique du sceptique, cet instinct de vérifier, de douter et de tenir les sorties à un standard exigeant, n'est pas un frein à l'adoption : c'est un pilier de la culture d'ingénierie, et exactement la capacité dont l'organisation a besoin pour ne pas glisser vers le *vibe engineering* décrit au chapitre 4.

Forcer l'adoption produit l'effet inverse de celui recherché : un déploiement imposé aux sceptiques génère de la conformité de façade, pas de l'adhésion. Les sceptiques convertis, eux, deviennent les meilleurs ambassadeurs, à condition de leur apporter une **preuve concrète, propre à leur métier** (un cycle raccourci, une reprise de travail évitée, un taux d'erreur stable ou en baisse) plutôt qu'un discours général (WRITER, *Enterprise AI Adoption* 2026).

Cette approche suppose enfin de traiter le **retour d'expérience sur les échecs comme une ressource d'apprentissage institutionnelle**, et non comme une collection de défaites individuelles. Partager ce qui n'a pas fonctionné (un cas d'usage abandonné, un agent qui dérape, une spécification mal cadrée) nourrit l'adhésion collective bien plus sûrement que la seule mise en avant des succès. C'est le pendant du constat posé au chapitre 4 : l'expérimentation ne crée de la valeur que si l'organisation se donne les moyens d'en lire les résultats, échecs compris.



Le sceptique comme dispositif qualité

Plutôt que de déployer auprès des seuls volontaires et de contourner les réfractaires, on les place en position de **validateurs critiques** : ce sont eux qui éprouvent les solutions, débusquent les angles morts et fixent le niveau d'exigence. Le scepticisme cesse d'être un coût de la transformation pour en devenir un garde-fou.

Avec les premiers adoptants, ceux qui s'emparent de l'IA et la poussent dans leurs équipes, ils forment un couple complémentaire : les uns entraînent l'adoption, les autres en éprouvent la solidité.

5.4. Un cadre pour piloter la transition

Aucun modèle prescriptif ne tient ici : la maturité et la culture de chaque organisation rendent toute recette universelle inopérante. Cinq principes d'action ressortent toutefois des sections précédentes.

1. Traduire l'architecture cible en topologie d'équipes. La section précédente en a posé le principe ; reste à l'outiller. Le binôme CTO–RH part des chaînes décrites aux chapitres 6 à 8 (PDL, SDD, AI IDP) pour en dériver la taille et la composition des équipes, puis consigne le résultat dans un artefact partagé : l'organigramme de compétences rattaché à l'architecture, tenu à jour à une cadence régulière.

2. Cartographier les compétences qui changent. Avant de réorganiser, il faut savoir ce qui se déplace : quelles compétences se déprécient et lesquelles prennent de la valeur, toujours par rapport à un usage et jamais dans l'absolu. La distinction entre orchestrateurs et garants de la qualité est ici déterminante : les deux profils ont leur place dans l'architecture cible. Dissoudre les silos ne fait pas disparaître les spécialités : la sécurité ou la fiabilité restent des compétences profondes, même quand les frontières de rôles s'effacent.

3. Repenser les trajectoires sans laisser personne sur le bord. Trois chantiers se dessinent. Valoriser l'expérience accumulée, d'abord, en mettant à jour les référentiels pour que l'expertise métier et la maîtrise de la revue deviennent des atouts mesurables. Reconstruire ensuite la filière junior-senior, via le double mentorat décrit au chapitre 4, où seniors et juniors nourrissent mutuellement leurs pratiques. Accompagner enfin les artisans du code, en leur ouvrant des trajectoires de spécialiste de la qualité ou de la sécurité plutôt qu'une mutation forcée vers l'orchestration.

4. Aligner les outils RH. Grilles d'évaluation, critères de recrutement, niveaux de carrière : aucun de ces artefacts ne peut rester en l'état. Les modèles de gestion des talents bâtis autour de spécialités cloisonnées ne capturent plus la valeur réelle des profils. La refonte prend des mois et doit commencer avant que les écarts ne deviennent ingérables.

5. Investir dans le contexte. Le travail des agents dépend de la qualité du contexte qu'on leur expose : documentation à jour, spécifications versionnées, architecture lisible, et explicitation des processus et des règles métier (l'humain porte l'implicite, là où l'agent exige l'explicite). Ce qui passait pour un bonus devient une infrastructure critique, que le SDD (chap. 7) et la Context Factory (chap. 8) formalisent. C'est ce qui détermine si la transformation IA tient dans la durée ou s'effrite sous la dette, technique comme humaine.

Aucun de ces principes n'est décisif pris isolément ; c'est leur combinaison, portée par le binôme CTO–RH, qui l'est. Les outils et les formations ne viennent qu'ensuite, une fois la cible organisationnelle clarifiée.

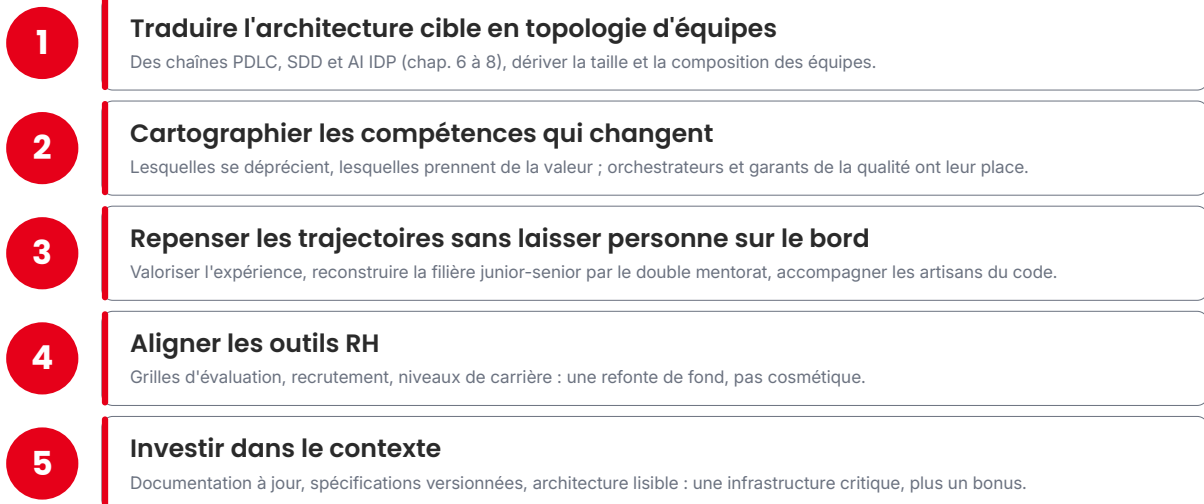
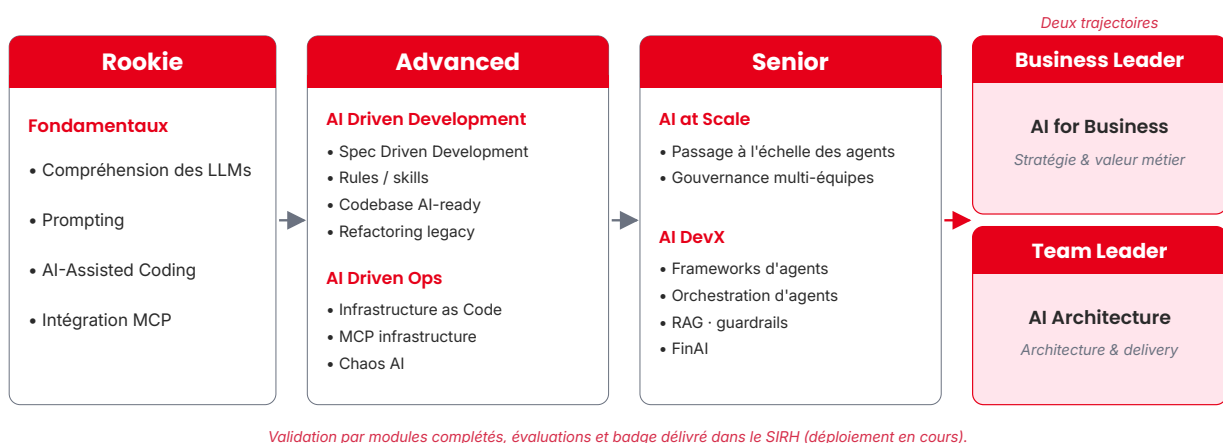


Figure 5. – Les cinq principes d'action pour piloter la transition, sous le pilotage conjoint du binôme CTO-RH.

Comment Zenika a construit son parcours IA Driven

Nous avons vécu de l'intérieur les ruptures décrites dans ce chapitre. Fin 2024, l'IA était un sujet d'expérimentation diffuse dans nos agences, sans cadre pour capitaliser les apprentissages ni les valoriser en mission. Nous avons répondu par un parcours « IA Driven » structuré en quatre paliers (Rookie, Advanced, Senior, Business ou Team Leader), piloté par un contrat entre la CxO Advisory Team, qui pose les standards, et les Directions d'agence, qui en assurent le déploiement local : socle commun prescriptif, mise en œuvre flexible.

L'enseignement principal tient moins à la structure qu'à ce qui la fait vivre. Les premiers adoptants, ceux qui s'emparent de l'IA et la poussent dans leurs missions, en sont les moteurs : ils contribuent aux contenus du parcours et signalent les écarts entre la théorie et l'usage réel. Sans cette communauté, le référentiel resterait théorique ; avec elle, il évolue en continu. Faire de ces profils des acteurs de la transformation, et pas seulement ses bénéficiaires, est la condition pour qu'un tel parcours tienne dans la durée.



Validation par modules complétés, évaluations et badge délivré dans le SIRH (déploiement en cours).

Figure 6. – Le référentiel de compétences « IA Driven » de Zenika : quatre paliers, des fondamentaux jusqu'à la divergence des trajectoires Business Leader / Team Leader.

PARTIE III

Ship: la fabrique du code

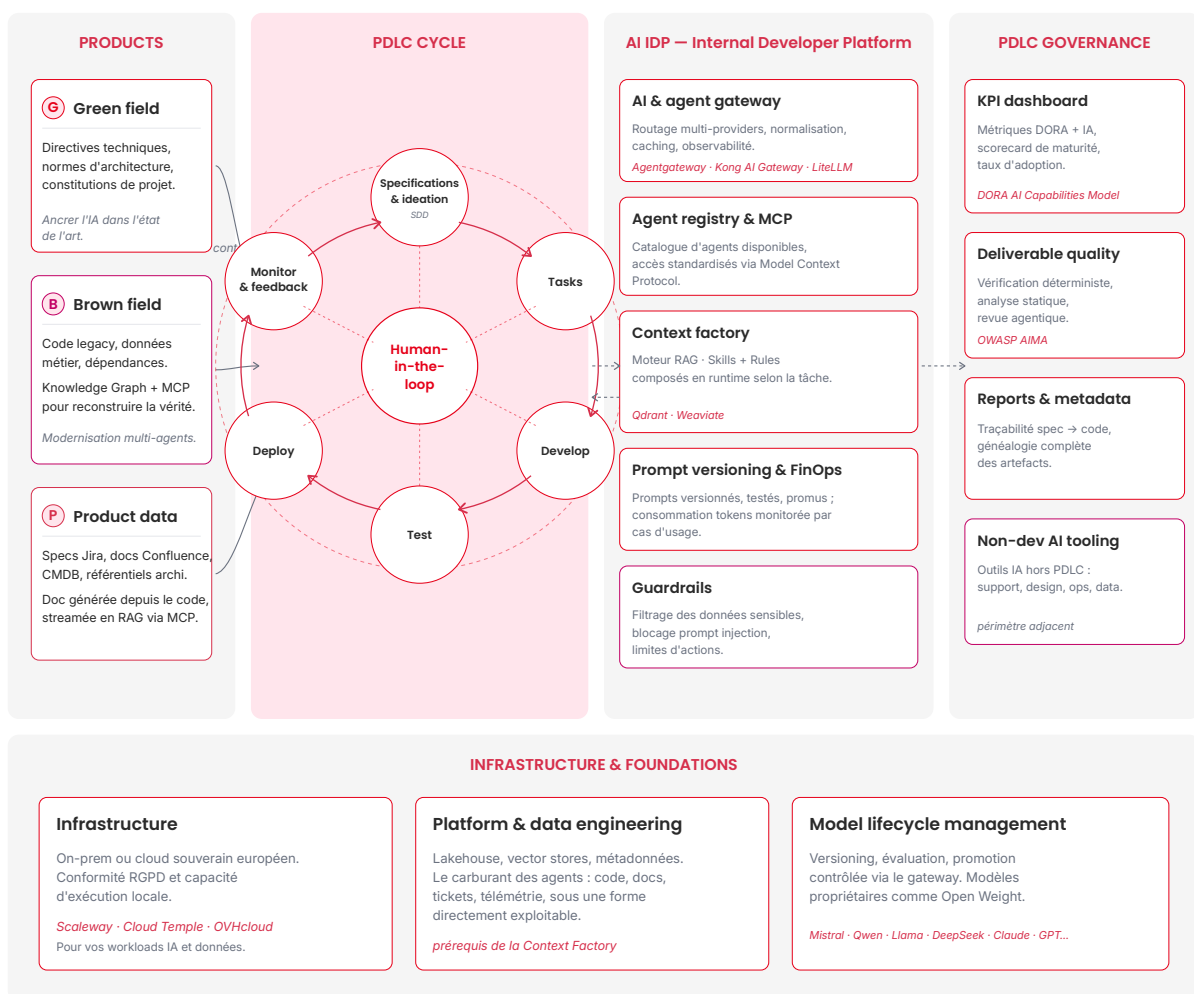


Chapitre 6 — Le cycle de vie du développement logiciel piloté par l'IA

L'IA Product Development LifeCycle (PDLC) redéfinit les processus et les rôles de la production logicielle.

Dompter l'agentique commence par cartographier ce qu'elle déplace : non pas un outil de plus dans l'IDE, mais une recomposition des **processus et des rôles** le long du cycle de production.

L'introduction de l'IA dans le cycle de vie du développement logiciel n'est pas une simple addition d'outil. C'est une recomposition de la chaîne de production : les artefacts longtemps tenus pour accessoires — la spécification, la documentation — (re)deviennent obligatoires, et les points de contrôle déterministes indispensables.



Lecture : les sources de contexte alimentent le cycle PDLC, orchestré par l'AI IDP, mesuré et contrôlé par la gouvernance, le tout sur des fondations souveraines.

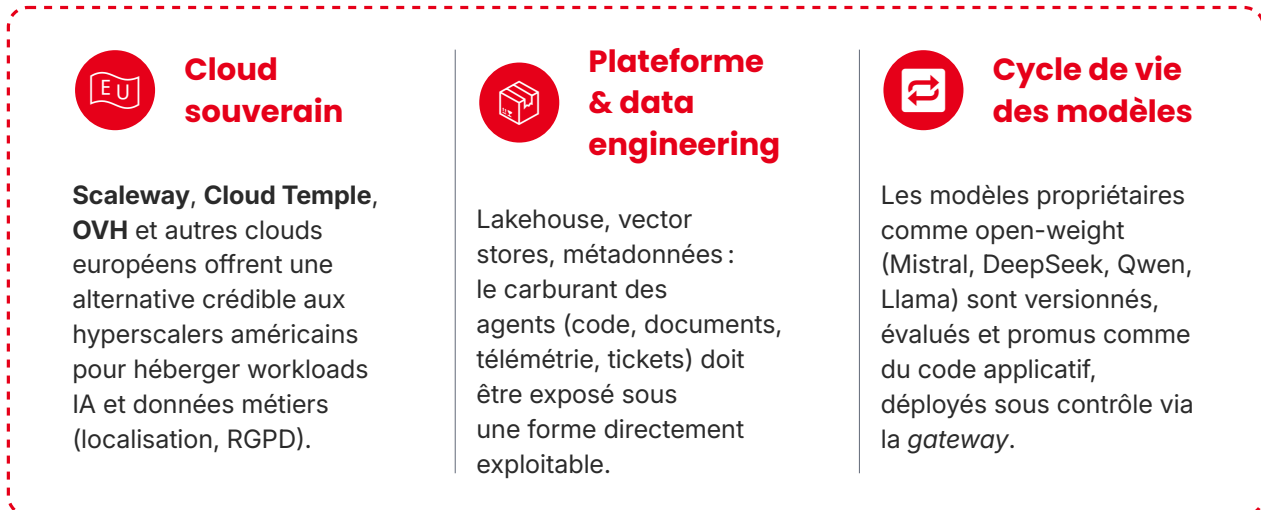
Figure 7. — Schéma fonctionnel de l'AI PDLC : produits (Green field, Brown field), cycle de développement avec human-in-the-loop, plateforme AI IDP (gateway, registry, context factory, prompt versioning, guardrails), gouvernance, et fondations infrastructure.

Le schéma articule quatre zones : les **sources de contexte** (green/brown field) alimentent le **cycle PDLC** orchestré par les agents, avec un **human-in-the-loop** dont le rôle se calibre : déterminant pour démarrer et maîtriser la transformation, il s'efface progressivement sur le code pur mais reste dans la boucle pour les décisions structurelles — c'est la proportion qui évolue ; l'**AI IDP**

fournit les briques d'orchestration (chapitre 8) ; la **gouvernance** mesure et contrôle (chapitre 10) ; l'**infrastructure** assure la souveraineté d'exécution, traitée ci-dessous.

6.1. Infrastructures et fondations

L'AI PDLC ne peut tenir ses promesses de souveraineté qu'à condition de reposer sur une infrastructure compatible avec les exigences réglementaires européennes et la possibilité d'exécution en local. Trois familles de briques composent ce socle.



6.2. Les cas d'usage du framework: Green Field vs Brown Field

Le framework structure l'approche selon le contexte que l'IA doit ingérer. Pour un projet **green field**, le contexte n'existe pas encore : l'ingénieur de contexte (l'architecte) ancre l'IA dans l'état de l'art en codifiant directives techniques, normes d'architecture (ADR, sécurité, observabilité) et constitution de projet — l'ensemble versionné avec le code sous forme d'**instructions de développement partagées**.

Pour un projet **brown field** — la majorité des cas en entreprise — l'enjeu n'est plus de définir un cap, mais de **reconstruire la vérité métier** à partir d'un système existant.

Au-delà du code, les **données produit** alimentent aussi le cycle : les spécifications sont généralement éparpillées (Jira, Confluence, CMDB, référentiels d'architecture) et la documentation se périmise. Trois chantiers data accompagnent l'adoption — nettoyer le patrimoine existant et nommer les Data Owners/Stewards manquants ; générer la documentation depuis le code et la versionner avec lui ; exposer ces sources aux agents en les streamant vers une capacité RAG accessible via MCP.



Deux mécanismes répondent à ce défi : les **Knowledge Graphs** extraient dépendances, schémas et règles métiers pour que l'IA navigue l'architecture réelle plutôt que d'inférer depuis quelques fichiers ; l'**analyse déterministe via MCP** donne aux agents accès à des outils d'analyse statique, de recherche ou d'exécution de tests, aux résultats vérifiables.

Sur cette base, des **architectures multi-agents** prennent en charge la modernisation — extraction des règles métier, spécification cible, génération, vérification de non-régression — chacun dans un contexte borné, sous contrôle humain.

Le défi du brown field

L'IA générative est entraînée sur du code public récent ; elle ne connaît rien du système legacy de l'entreprise. Sans contexte structuré, elle hallucine. Avec un contexte mal cadré, elle propage la dette.

6.3. Le différenciateur Zenika

Face à un marché fragmenté entre outils propriétaires et approches partielles, Zenika propose une approche intégrée qui combine cinq piliers cohérents : la méthodologie produit des artefacts vérifiables, le framework les rend portables, les pratiques industrialisées en assurent la diffusion à l'échelle, un CI/CD enrichi installe les boucles de vérification déterministe (quality gates IA, traçabilité spec ↔ artefact) au cœur du pipeline de delivery, et une boucle produit complète ramène les signaux d'usage jusque dans la spécification.

Une application moderne à l'état de l'art doit répondre à des contraintes non fonctionnelles importantes comme l'efficacité, la performance, la compatibilité, l'accessibilité. Ce résultat n'est pas garanti par défaut et doit être intégré dans les outils à travers les spécifications et skills, et contrôlé avec des indicateurs.



Méthodologie structurée

Le SDD redéfinit le cycle de développement autour de la spécification comme artefact central, vérifiable et auditable — pas un simple optimiseur de prompts.



Framework modulaire

L'AI IDP — gateway, registry, context factory — garantit la liberté de basculer à tout moment, sans refonte du delivery.



Pratiques industrialisées

Standards diffusés via des **instructions de développement partagées** versionnées dans le code, et des **Skills** définis au niveau de l'équipe.



CI/CD enrichi

Pipeline complétée de **processus asynchrones** (revue agentique, alignement spec ↔ code) qui maintiennent la cohérence dans la durée.



Boucle produit complète

Du recueil de feedback (télémétrie, verbatim) au déploiement augmenté (feature flags, canary release, rollback piloté), les agents alimentent les spécifications, ce qui distingue les pratiques IA d'entreprise d'un simple outillage développeur.

Chapitre 7 — Le Specification Driven Development (SDD)

La spécification comme contrat exécutable, pour maîtriser la qualité de production des agents.

Cartographier les processus ne suffit pas si chaque génération reste un acte non reproductible : **pour dompter l'agentique à l'échelle, il faut une discipline qui rende la qualité prévisible** — c'est l'objet du SDD.

Une filiation : BDD, ATDD

Le SDD prolonge l'héritage du **BDD** (*Behavior-Driven Development*, Dan North, 2003) et de l'**ATDD** (*Acceptance Test-Driven Development*) : la spécification exécutable y était déjà le contrat partagé entre métier et technique. L'apport du SDD à l'ère agentique : elle devient la **source unique de génération** du code par les agents, plus seulement un test de validation.

7.1. L'échec du « Vibe Coding »

Les pratiques dominantes — le *Vibe Coding* — reposent sur une interaction intuitive entre développeur et IA : prompt informel, génération, ajustement manuel. À l'échelle du prototype, ça suffit ; à l'échelle industrielle, chaque interaction est unique et non reproductible — la qualité dépend de l'expertise individuelle, les standards ne sont pas partageables, la vérification repose sur l'intuition. **Le vibe coding mène au mur de la maintenance.**

7.2. L'avènement du SDD



Thoughtworks Technology Radar

Le radar Thoughtworks Vol. 34 (avril 2026) confirme la trajectoire : *Curated shared instructions for software teams* passe en **Adopt** — aligné sur notre principe d'**instructions de développement partagées**. Le SDD, identifié dès le Vol. 33, reste en *Assess* avec un niveau de maturité croissant : la spécification s'impose comme **contrat exécutable** qui précède et valide la génération de code.

Le SDD élève la spécification au rang de **source de vérité contraignante** — **le code n'est plus l'artefact premier**. Birgitta Böckeler (*SDD: the 3 tools*, Martin Fowler, 2025) en distingue trois postures qui jalonnent l'adoption.



Spec-first

La spécification est rédigée d'abord et utilisée comme prompt structuré pour générer le code. Mais elle n'est pas maintenue dans la durée : elle peut diverger silencieusement du code ensuite édité à la main.



Spec-anchored

Spec et code synchronisés en permanence. Le code reste éditable, mais toute évolution est ancrée dans une spécification mise à jour ; les divergences sont détectées.



Spec-as-source

Seule la spécification est éditée. Le code est un artefact généré, comparable au binaire d'un compilateur ; modifier la spécification suffit à régénérer le système.

C'est la posture vers laquelle nous tendons chez Zenika. Viser le **spec-as-source** oriente l'organisation et les méthodes vers une **qualité intransigeante** : la spécification devient la source unique d'audit, de revue et de raisonnement. Cette montée en exigence se fait **graduellement** selon la maturité des équipes — démarrer en *spec-first*, consolider en *spec-anchored*, puis viser le *spec-as-source*. Mais à l'image de l'open-source, il reste **obligatoire et possible de pouvoir rentrer dans le code**, le lire et le comprendre : la lisibilité du code généré n'est pas négociable — sécurité, auditabilité, conformité l'exigent. Ce sont les outils du **craftsmanship** — lints, tests de tous types, formatage automatique... — qui garantissent la maintenabilité du code et de l'architecture, là où la spécification garantit celle du produit. Ces deux disciplines, indissociables, rendent le système maintenable dans la durée, quel que soit son mainteneur futur — humain ou IA.

7.3. Outillage: de l'idéation à l'orchestration

L'écosystème SDD s'organise autour d'un noyau d'outils complémentaires. **GitHub Spec Kit** propose une structure ouverte pour formaliser les artefacts dans le repository, avec un cycle clair *spec* → *plan* → *tasks* → *implementation* extensible aux projets brownfield. À titre d'exemples, **OpenSpec** apporte une logique de *delta* de spec versionné, pratique pour traiter chaque évolution comme un changement explicite ; **BMAD** formalise des personas d'agents (PM, Architecte, Dev...) collaborant via un workflow structuré.

Au-delà de l'outil, le SDD impose une discipline : chaque évolution est formulée comme un *delta* de spécification (intention, exigences impactées, scénarios à vérifier) avant la génération de code — d'abord ce qui change, ensuite comment c'est implémenté. Cette séquence rend la revue croisée entre humains et agents praticable, et conserve l'historique décisionnel de chaque modification — un travail que **Zenflow** prolonge avec ses **agents dédiés vers le spec-as-source**.



Zenika Zenflow

Zenflow (open-source) matérialise nos pratiques agentiques en starters réutilisables : compatibilité **multi-outils** (Copilot, OpenCode, Claude Code), **multi-stacks** back/front (Java/Spring, Python/FastAPI, Go/Gin, React, Next.js) et un **workflow d'orchestration** éprouvé (un agent *Orchestrateur* qui délègue à des sous-agents spécialisés). Init en deux commandes, artefacts versionnés dans le repository — strictement aligné sur le principe des instructions de développement partagées.

Chapitre 8 — L'AI IDP (Internal Developer Platform)

La couche d'orchestration qui découple workflow et fournisseur.

Une fois la méthode posée (SDD), encore faut-il l'outiller. L'AI Internal Developer Platform (AI IDP) est la plateforme qui industrialise cette discipline et **fait basculer l'agentique d'un coût subi à un actif piloté**. Elle se positionne entre les outils de développement (agents de code, assistants spécialisés) et les fournisseurs de modèles (APIs cloud ou modèles locaux), et orchestre l'ensemble du cycle d'usage de l'IA dans l'organisation.

Un cycle produit agentique, pas seulement développeur

Le PDLC dépasse le code. En amont, l'IA accélère la **discovery produit**, la rédaction des *user stories* et la priorisation du *backlog* ; les rituels agiles deviennent des points de synchronisation entre humains et agents. En aval, le **déploiement augmenté** (feature flags pilotés par agents, *canary release* observé, rollback automatisé) et le **recueil de feedback** (télémétrie produit, verbatim utilisateur, NPS) ferment la boucle vers la spécification grâce à l'agentique. L'AI IDP outille la chaîne technique ; ce sont les agents IA d'entreprise qui orchestrent l'ensemble — produit, projet, delivery, ops.

Aucun produit unique ne couvre aujourd'hui l'intégralité du périmètre. L'AI IDP s'implémente par **assemblage** de quatre briques complémentaires, chacune débrayable et remplaçable indépendamment des autres.

8.1. AI Gateway: routage et contrôle

L'AI Gateway centralise le trafic LLM. Tous les appels des agents, IDE et services internes vers les modèles transitent par cette brique, ce qui permet de gérer dans un seul endroit l'authentification, le routage multi-providers, le caching sémantique, le rate limiting, l'observabilité (latence, coût, succès), et la conformité (filtrage des données sensibles).

Côté implémentations, **Agentgateway** (orienté trafic agent-à-agent et MCP), **Kong AI Gateway** (extension de la gateway Kong avec plugins LLM) et **LiteLLM** (proxy léger compatible OpenAI/Anthropic/Bedrock/locaux) sont des références ouvertes. Le découplage opéré ici est fondamental : changer de fournisseur de modèle n'a aucun impact sur les outils en aval, puisque tous adressent la gateway et non les APIs des fournisseurs.

8.2. Agent Registry et MCP

L'Agent Registry catalogue les capacités des agents disponibles dans l'organisation : leurs rôles, les modèles qu'ils requièrent, les autorisations dont ils disposent, l'historique de leur usage. Il joue pour les agents le rôle qu'un *service registry* joue pour les microservices.

À ce catalogue s'ajoute la standardisation des accès aux données et aux outils via le **Model Context Protocol (MCP)**. MCP normalise la manière dont un agent expose et consomme des sources de contexte : repository de code, base de connaissances, ticketing, télémétrie, outils d'analyse. Une intégration MCP est réutilisable d'un agent à l'autre, d'un outil à l'autre.

Cette combinaison registry + MCP transforme un écosystème d'agents disparates en plateforme cohérente. Elle conditionne la capacité à déployer rapidement de nouveaux cas d'usage sans réinvention permanente des intégrations.

8.3. Context Factory

La Context Factory est le **moteur RAG** de la plateforme : elle alimente les agents avec le contexte métier exact dont ils ont besoin pour produire un travail pertinent.

Concrètement, elle indexe les sources internes (code, documentation, tickets, runbooks, schémas de données) dans une base vectorielle — **Qdrant**, **Weaviate** ou équivalent — et expose des points d'accès permettant à un agent d'interroger sémantiquement ce corpus en temps réel. Les *skills* et *règles* spécifiques à un domaine (sécurité, accessibilité, FinOps, performance) y sont versionnés et composés à la volée selon le contexte de la tâche.

L'enjeu est double. D'une part, **réduire les hallucinations** en ancrant chaque génération sur des faits du système réel plutôt que sur les a priori du modèle. D'autre part, **diffuser les standards** : un agent qui interroge la Context Factory bénéficie automatiquement des règles partagées de l'entreprise, sans qu'il soit nécessaire de les réinjecter dans chaque prompt. Le **context engineering** s'impose d'ailleurs comme une pratique de premier plan — la fenêtre de contexte est désormais une *design surface* à part entière, plus une simple variable de prompt.

Zenika Context Nexus — structurer ce que la Context Factory distribue

La Context Factory est aussi performante que le contexte qu'elle indexe. Indexer une documentation dispersée et non structurée revient à distribuer du bruit aux agents plutôt que des standards partagés.

Le **Context Nexus** est le framework documentaire proposé par Zenika pour combler ce manque. Il organise la connaissance organisationnelle en quatre registres — **Intent** (les décisions et contraintes de l'organisation), **Knowledge** (la mémoire institutionnelle), **Contracts** (les engagements vérifiables) et **Skills** (les manières de faire) — conçus pour être exploitables par les agents IA et maintenus en continu par le workflow de développement lui-même. Sans cette matière structurée, la Context Factory indexe sans discernement, et les agents génèrent sans fil directeur. Le chapitre 9 présente ce framework et sa progression d'adoption.

8.4. Prompt Versioning & FinOps

Les prompts ne sont pas des chaînes de caractères jetables — ce sont des **artefacts d'ingénierie** qui méritent le même soin que du code applicatif.



Versioning

Chaque prompt système, chaque template d'agent est versionné, testé sur des jeux de cas canoniques, et promu de manière contrôlée d'un environnement à un autre. Une régression de qualité doit être détectée avant production, pas découverte par les utilisateurs.



FinOps

Le coût d'usage de l'IA est suivi par agent, par équipe, par cas d'usage. Des alertes et quotas évitent les dérapages budgétaires liés à un agent en boucle, à un contexte trop long, ou à un usage non prévu.



Guardrails

Des politiques de sécurité — filtrage des données sensibles, blocage des prompts injectés, limitation des actions à risque — sont appliquées en entrée et en sortie des appels modèles. Elles protègent à la fois l'organisation (fuite de propriété intellectuelle) et les applications utilisatrices (injection, exfiltration).

La maîtrise des coûts en tokens (FinOps) permet également de limiter les impacts environnementaux (GreenOps) associés à l'utilisation de ces outils, celle-ci étant corrélée à l'utilisation de ressources numériques.

Divers moyens peuvent être utilisés pour limiter ces coûts : * Choisir le bon algorithme pour chaque tâche * Choisir des modèles plus petits et hébergés en France, voire sur la machine locale, tant que les performances sont bonnes * Utiliser des techniques de prompt engineering, mutualiser les réponses si possible * Limiter les tokens envoyés et générés à travers diverses techniques

Cette quatrième brique ferme la boucle de l'AI IDP : sans gestion versionnée des prompts, sans contrôle des coûts et sans guardrails, la plateforme reste exposée aux dérives. Et c'est cet assemblage débrayable, où chaque brique évolue indépendamment, qui produit la souveraineté opérationnelle promise au chapitre 2.

8.5. Industrialiser au Run

Reste l'échelle d'exécution. En production (le *Run*), l'enjeu se déplace vers l'industrialisation des workflows agentiques : orchestrer les agents, dimensionner les ressources de calcul (GPU) et, surtout, observer les coûts et les performances en continu, sans quoi un parc d'agents devient impossible à piloter. Des briques comme **OpenShift** et **vLLM** offrent une base pour cette scalabilité, en combinant orchestration des charges agentiques et observabilité fine de l'inférence (latence, débit, coût par requête). Ces technologies ne se déploient pas seules : elles exigent une expertise pointue en données et en plateforme pour être exploitées efficacement.

C'est tout l'objet d'une *AI Factory* industrialisée, où une gateway centralisée, un moteur d'inférence dimensionné et la traçabilité des prompts et des raisonnements rendent l'exécution auditable. L'observabilité instaurée à ce niveau alimente directement les métriques de gouvernance du chapitre 10.



Chapitre 9 — Le Context Nexus: la documentation comme infrastructure agentique

Ce que les agents savent dépend de ce que l'organisation documente.

9.1. Le paradoxe du contexte

La pertinence d'un agent IA est plafonnée par la qualité du contexte qu'on lui fournit. Le modèle choisi, propriétaire ou open weight, local ou cloud, est rarement le facteur limitant. Ce qui détermine la qualité de la génération, c'est la qualité du contexte injecté : un agent puissant sans contexte structuré hallucine ou, pire, propage la dette existante en la rendant plus difficile à détecter. Le §6.2 l'a posé pour le brown field ; le principe 5 du §5.4 l'a formulé comme impératif organisationnel. Ce chapitre en est la réponse concrète.

Cette réalité déplace la documentation d'une bonne pratique d'ingénierie vers une infrastructure de production. Tant que la documentation reste un effort volontaire, géré après coup, elle n'est pas fiable comme source de contexte agentique. Le Context Nexus est un framework qui traite ce problème à la racine : non pas en ajoutant de la discipline documentaire, mais en organisant la connaissance organisationnelle pour qu'elle soit exploitable par les agents, et maintenue vivante par le workflow de développement lui-même.

9.2. Quatre registres, une organisation

Le Context Nexus organise la connaissance organisationnelle en quatre registres. Chacun répond à une question distincte, implique un propriétaire différent, et se consomme différemment dans la chaîne agentique.

Intent — ce que l'organisation décide. Les directives, les standards, les contraintes non négociables : règles d'architecture, politiques de sécurité, conventions de développement. Ce registre est la voix de l'organisation dans chaque session de génération. Il est permanent, prescriptif, et ne peut être mis à jour que par des humains.

Knowledge — ce que l'organisation sait. Les décisions passées et leur contexte, les apprentissages issus des incidents, les règles métier implicites qui ne figurent nulle part ailleurs. C'est la mémoire institutionnelle : ce qui permettra à un agent de ne pas répéter une erreur déjà commise, de comprendre pourquoi un choix d'architecture a été fait plutôt que comment il a été fait.

Contracts — ce que l'organisation garantit. Les assertions vérifiables : seuils de qualité, critères d'acceptation, SLO, définitions du terminé. Un contrat n'est pas une intention — c'est une frontière avec un seuil mesurable. Ce registre permet à un agent de savoir non pas ce qu'il doit faire, mais à partir de quand ce qu'il produit est acceptable.

Skills — comment l'organisation travaille. Les procédures, les patterns récurrents, les recettes opérationnelles. Ce que tout nouveau membre apprendrait d'un collègue expérimenté en quelques semaines : les manières de faire propres à l'équipe ou à l'organisation.



L'invariante du Context Nexus

Intent et Contracts ne sont jamais délégués à un agent. Ils définissent ce que l'organisation décide et ce qu'elle garantit : les modifier sans décision humaine explicite reviendrait à laisser l'IA redéfinir les engagements de l'organisation à sa place.

Knowledge et Skills, en revanche, peuvent progressivement bénéficier d'une assistance agentique : un agent peut proposer qu'un incident récent génère une nouvelle entrée dans Knowledge, ou qu'un pattern émergent soit formalisé dans Skills. La décision finale reste humaine ; l'effort de rédaction peut ne plus l'être.

9.3. Du contexte individuel au contexte partagé

Le premier bénéfice du Context Nexus est individuel : un développeur ou une développeuse qui structure son contexte local produit une IA qui se comporte de manière cohérente avec ses intentions. C'est utile, mais ce n'est pas là que réside la valeur organisationnelle.

La valeur apparaît quand le contexte devient partagé. Quand l'Intent d'une organisation descend dans chaque session de génération de chacune de ses équipes, ces équipes produisent un travail cohérent entre elles, sans coordination supplémentaire. Quand le Knowledge est commun, un agent sait ce que l'organisation a appris, pas seulement ce qu'une équipe a vécu. C'est à cette échelle que le Context Nexus change la nature du problème.

Le modèle de partage s'organise en hiérarchie : l'organisation définit un socle commun, les équipes l'héritent et l'affinent à leur périmètre. La règle est stricte : une équipe étend le contexte organisationnel, elle ne le contredit pas silencieusement. Si une équipe a une raison légitime de s'écarter d'un standard, cette exception doit être documentée explicitement, avec sa justification. L'écart devient alors une décision traçable plutôt qu'une divergence invisible.

Cette logique rejoint directement la loi de Conway évoquée au chapitre 3 : les systèmes logiciels reflètent la structure de communication de l'organisation qui les produit. Le Context Nexus en est le pendant documentaire : il reflète la structure de connaissance de l'organisation. Si cette structure est cohérente, les agents produisent un travail cohérent. Si elle est fragmentée, aucun modèle ne compensera ce que l'organisation n'a pas su formaliser.

9.4. Le Context Nexus dans le PDLC

Le Context Nexus n'est pas un artefact à construire à part, avant de revenir au vrai travail. Il se constitue en continu, comme sous-produit du cycle de développement.

Une décision d'architecture va dans Knowledge. Une règle issue d'un incident de production va dans Contracts. Une nouvelle convention d'équipe va dans Skills. La spécification d'une fonctionnalité, une fois validée, alimente Intent. Chaque étape du PDLC décrit aux chapitres 6 et 7 génère naturellement du contexte : la question est seulement de savoir si ce contexte est capturé et structuré, ou s'il reste dans les échanges informels et les mémoires individuelles.

La Context Factory (§8.3) est le moteur de distribution : elle indexe, récupère et compose le contexte à la demande des agents. Le Context Nexus en est la matière : sans lui, la Context Factory n'a rien à distribuer.



Le cycle de vie du contexte mérite d'être piloté comme celui d'un artefact de code : chaque entrée porte une date de création, un propriétaire identifié par son rôle, et une date de révision attendue. Un contexte qui vieillit sans être révisé devient une source de génération incorrecte — l'équivalent d'une dépendance non mise à jour dans un projet. La gouvernance du Nexus n'est pas une discipline supplémentaire ; c'est une conséquence naturelle du pipeline CI/CD, où certains événements (une mise en production, une décision d'architecture, un post-mortem) déclenchent une révision des artefacts concernés.

9.5. Se projeter : trois étapes

Le Context Nexus suit une progression naturelle. Aucune organisation ne démarre avec un système complet.

La première étape est la plus accessible : quelques fichiers versionnés dans Git, organisés selon les quatre registres. Pas d'infrastructure supplémentaire. Les agents de code consomment ces fichiers directement depuis le dépôt. Cette mise en place couvre l'essentiel de la valeur pour une équipe en phase d'adoption.

La deuxième étape ajoute la dimension organisationnelle : un socle partagé au niveau de l'organisation, des équipes qui l'héritent et le complètent à leur niveau, un processus léger de révision intégré au workflow de développement. À ce stade, la cohérence de la génération entre équipes commence à être mesurable.

La troisième étape industrialise la distribution : indexation vectorielle, MCP pour les Skills, Context Assembler qui compose le contexte à la volée selon la nature de la tâche. C'est à ce stade que la Context Factory du chapitre 8 atteint son potentiel.



Pour aller plus loin

L'implémentation détaillée du Context Nexus — guide d'implémentation technique, progression par étapes, intégration avec les agents MCP — est disponible sur cnexus.fr.

PARTIE IV

Mesure & pilotage





Chapitre 10 — Gouvernance du PDLC et métriques

Garantir le ROI d'un écosystème complexe : boucles de rétroaction, conformité et pilotage par la donnée.

L'ingénierie logicielle assistée par IA exige un système nerveux : des **boucles de rétroaction** mesurables qui transforment l'intuition en pilotage, et des **garde-fous de conformité** qui rendent les choix techniques défendables face à la régulation. La gouvernance du PDLC n'est pas une couche bureaucratique posée sur le delivery — c'est la condition pour que les gains individuels mesurés au chapitre 1 se transforment en performance organisationnelle durable.

10.1. Une gouvernance fédérée : aligner sans recentraliser

Sans pilotage d'ensemble, les usages IA restent au stade individuel. Les initiatives se multiplient dans les équipes et les lignes produit, mais demeurent dispersées, sans capitalisation ni standardisation à l'échelle de l'entreprise. Ce silotage est le pendant organisationnel du « pilote permanent » décrit au chapitre 3 : beaucoup d'expérimentations, peu de passage à l'échelle.

Le phénomène s'aggrave en 2026 avec le volume de cas d'usage. La plupart des modèles de gouvernance n'ont pas été conçus pour cette charge : les comités de revue centralisés saturent et deviennent des goulots d'étranglement (Eficode, *State of AI in Software Organizations* 2026). Tout recentraliser est illusoire ; tout laisser faire produit de la dispersion.

La réponse tient dans une **gouvernance fédérée** : centraliser la stratégie (données et IA, standards, conformité, socle de sécurité) et décentraliser l'exécution au sein de plateformes dédiées à chaque ligne de produits. Le centre fixe le cadre commun et les garde-fous ; les équipes produit gardent l'autonomie d'exécution à l'intérieur de ce cadre. C'est le modèle que nous avons éprouvé en interne avec le parcours IA Driven (chapitre 5) : un contrat entre une équipe transverse qui pose les standards et des directions qui en assurent le déploiement local, soit un socle commun prescriptif et une mise en œuvre flexible.

Cette articulation se mesure. L'indice de diffusion des instructions de développement partagées et des Skills, l'un de nos indicateurs clés présentés ci-après, objective si les standards descendent réellement jusqu'aux équipes ou s'ils restent théoriques.

10.2. Mesurer le delivery : DORA et ses extensions

Les métriques DORA (Deployment Frequency, Lead Time for Changes, Change Failure Rate, Time to Restore Service) restent le référentiel de mesure de la performance du delivery. Le programme s'enrichit en 2025 d'un **AI Capabilities Model** dédié, qui ajoute des capacités spécifiques au développement assisté ; puis, en 2026, de la modélisation de la trajectoire d'adoption en **J-Curve** : un creux temporaire — le « coût de scolarité » de la transformation, dû au *learning curve*, à la *verification tax* et à l'adaptation des pipelines — précède la croissance exponentielle.¹

¹Ces évolutions récentes (AI Capabilities Model, modélisation en J-Curve) traduisent un certain engouement pour l'outillage IA, là où DORA nous avait jusqu'ici surtout fourni des analyses de pratiques issues du terrain — un déplacement que nous gardons à l'œil.

Cette J-Curve, nous la constatons sur le terrain. Chez nos clients, l'adoption de l'IA agentique s'accompagne systématiquement d'un creux de 6 à 12 mois dont les composantes recoupent précisément les segments chiffrés au chapitre 2 (figure 2) : risque méthodologique (dont le tuning), infrastructure à mettre en place, montée en compétence MLOps. Le suivi DORA en continu transforme ce creux attendu en **investissement piloté** plutôt qu'en surprise budgétaire — et permet d'éviter le piège qui consiste à interrompre l'effort pendant la phase d'apprentissage en l'interprétant comme un échec.

Au-delà des métriques canoniques, le pilotage du framework Zenika s'appuie sur des indicateurs propres qui révèlent la valeur réelle de l'IA — là où DORA seul peut masquer une dérive.

Les indicateurs clé Zenika pour l'IA

Taux de vérification	Pourcentage de code IA validé par des tests déterministes avant merge.
Ratio spécifications/code	Proportion d'artefacts SDD par rapport au code généré directement.
Score de souveraineté	Capacité mesurée à basculer entre providers IA sans impact delivery.
Indice de diffusion	Adoption des instructions de développement partagées et Skills à l'échelle de l'organisation.

10.3. Sécurité et conformité: OWASP AIMA

La sécurité du PDLC assisté par IA introduit de nouvelles surfaces d'attaque : *prompt injection*, exfiltration de données via les outils, supply chain compromise sur les modèles, *data poisoning* du contexte. Ces risques exigent un référentiel adapté.

Le framework **OWASP AIMA** (AI Maturity Assessment) propose une grille structurée pour évaluer la maturité sécurité d'une organisation sur ses usages IA. Couplé aux contrôles classiques (analyse statique, scanners de dépendances, revue agentique), il permet de positionner la sécurité IA dans une démarche d'amélioration continue plutôt que dans une logique de checklist ponctuelle.



La règle stricte de la vérification

Aucun code généré par IA ne passe en production sans avoir franchi la **vérification déterministe** : analyse statique, tests unitaires et de propriété, linters, revue agentique distincte de l'agent générateur. C'est la réponse concrète au constat que seuls 48 % des développeurs vérifient systématiquement le code avant commit (Sonar 2026).



10.4. Le pipeline « AI-augmented CI/CD »

L'intégration de cette gouvernance dans les pipelines existants (GitHub Actions, GitLab CI) se fait par ajout d'étapes — agent de spécification, génération sous contexte, *quality gates* déterministes, revue agentique distincte de l'agent générateur, revue humaine finale — sans remplacement de l'existant. Chaque étape est un *quality gate* : si un seuil n'est pas atteint, le pipeline s'arrête et l'agent est relancé avec le feedback de l'étape en échec. Cette boucle automatique réduit le temps de correction et matérialise, concrètement, ce qu'est une gouvernance du PDLC à l'ère de l'IA.

Conclusion — Checklist de maturité IA pour le CTO

Le développement assisté par IA agentique n'est plus une option : c'est une transformation inévitable. La question n'est plus de savoir si votre organisation adoptera l'IA dans son cycle de développement, mais comment elle le fera — de manière opportuniste et dépendante, ou de manière méthodique et souveraine.

11.1. Les 7 questions à vous poser

Pour évaluer votre niveau de préparation, répondez à ces sept questions.

1. Vos équipes utilisent-elles un cadre méthodologique structuré (SDD) ?
2. Votre outillage est-il découplé du fournisseur de modèle via un **AI IDP** (gateway, registry, context factory, prompt versioning) ?
3. Pouvez-vous basculer vers des modèles Open Weight locaux en moins de 48 h sans impact sur le delivery ?
4. Vos exigences sont-elles codifiées dans des **instructions de développement partagées** versionnées au sein du code ?
5. Chaque ligne de code IA passe-t-elle par une **vérification déterministe** avant merge (tests, analyse statique, revue agentique) ?
6. Vos projets *brown field* disposent-ils d'un contexte structuré (Knowledge Graph, MCP) avant toute génération ?
7. Mesurez-vous les métriques DORA augmentées (taux de vérification, ratio spécifications/code, score de souveraineté) en continu ?



Lecture du score

0-2 « oui » — Urgence stratégique. Risque de rejoindre les 40 % de projets d'IA agentique que Gartner prévoit abandonnés d'ici fin 2027.

3-5 « oui » — En bonne voie, structurez. Les fondations existent ; il manque la cohérence d'ensemble.

6-7 « oui » — Excellence, industrialisez. Vous êtes en mesure de devenir un référent dans votre écosystème.

11.2. Passer à l'action

La méthodologie et le framework existent

Il ne manque que la décision de les adopter. Zenika vous accompagne à chaque étape de cette transformation.

Contactez-nous pour un diagnostic de maturité IA de votre organisation.

Annexes

12.1. Sources et bibliographie

12.1.1 Rapports et études

- DORA — *The ROI of AI-Assisted Software Development 2026*, Google Cloud. <https://cloud.google.com/resources/content/dora-roi-of-ai-assisted-software-development-2026>
- DORA — *State of AI Assisted Software Development 2025*, Google Cloud. <https://cloud.google.com/resources/content/2025-dora-ai-assisted-software-development-report>
- SonarSource — *State of Code Developer Survey 2026* (72 % d'usage quotidien parmi ceux qui ont essayé ; 96 % ne font pas pleinement confiance ; 48 % vérifient avant commit ; 42 % du code committé est généré par l'IA, projeté à 65 % en 2027 ; 35 % via des comptes personnels). <https://www.sonarsource.com/resources/developer-survey-report/>
- Stack Overflow — *Developer Survey 2025* (84 % utilisent/prévoient l'IA ; 29 % de confiance, -11 pts ; 46 % de défiance envers la justesse ; 45 % frustrés par les solutions « presque correctes »). <https://survey.stackoverflow.co/2025/ai/>
- Eficode — *State of AI in Software Organizations 2026* (plus de 270 organisations, mars 2025–mars 2026 ; adoption en largeur mais pas en profondeur). <https://www.eficode.com/guides/how-software-organizations-are-moving-from-ai-pilots-to-ai-transformation>
- WRITER — *Enterprise AI Adoption 2026* (un déploiement imposé produit de la conformité, pas de l'adhésion ; conversion des sceptiques par la preuve métier). <https://writer.com/blog/enterprise-ai-adoption-2026/>
- NVIDIA — *State of AI 2026* (88 % des organisations constatent une hausse de CA liée à l'IA, dont 30 % > 10 % ; enquête août–déc. 2025). <https://blogs.nvidia.com/blog/state-of-ai-report-2026/>
- Exceeds AI — *AI Coding Assistant Adoption Rates 2026* (plus de 90 % des équipes d'ingénierie, contre 61 % un an plus tôt). <https://blog.exceeds.ai/ai-coding-tools-adoption-rates/>
- McKinsey — *The State of AI in 2025: Agents, Innovation, and Transformation*. <https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai>
- McKinsey — *Unlocking the Value of AI in Software Development*. <https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/unlocking-the-value-of-ai-in-software-development>
- Gartner — *40% of Enterprise Applications Will Feature Task-Specific AI Agents by 2026*. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025>
- Thoughtworks — *Technology Radar Vol. 34*, avril 2026. <https://www.thoughtworks.com/radar/>
- MIT Technology Review — *Everyone Wants AI Sovereignty. No One Can Truly Have It*. <https://www.technologyreview.com/2026/01/21/1131513/everyone-wants-ai-sovereignty-no-one-can-truly-have-it/>
- Ember — *Global Electricity Review 2025* (intensité carbone de l'électricité : ≈ 445 gCO₂/kWh en moyenne mondiale, ≈ 213 g pour l'UE, ≈ 560 g en Chine ; France très basse grâce au nucléaire). <https://ember-energy.org/latest-insights/global-electricity-review-2025/>
- Agence Internationale de l'Énergie (IEA) — *Energy and AI / Electricity 2025* (≈ 945 TWh consommés par les datacenters dans le monde en 2030, contre 415 en 2024). <https://www.iea.org/reports/energy-and-ai>
- The Shift Project — projections d'impact du numérique et de l'IA (scénario pessimiste : 1500 TWh et 920 MtCO₂e/an). <https://theshiftproject.org/>



- Mistral — *Sovereign AI in Europe*, White Paper. <https://europe.mistral.ai/whitepaper.pdf>
- Union européenne — AI Act. <https://artificialintelligenceact.eu/>
- Birgitta Böckeler — *SDD: the 3 tools* (blog Martin Fowler). <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- Randstad Digital — Michael Morris, *AI investment vs workforce readiness*, mai 2026 (échec de l'IA d'entreprise « à la couche d'implémentation »). <https://www.prnewswire.com/news-releases/new-randstad-digital-report-reveals-a-widening-disconnect-between-ai-investment-and-workforce-readiness-302768812.html>
- Alan (Emma Goldblum) — *Stop Testing Engineers Like It's 2015*, oct. 2025. <https://medium.com/alan/stop-testing-engineers-like-its-2015-why-we-embraced-ai-in-our-interviews-a21adec28a4f>
- Builder.io — *The Person Closest to the Problem*, 2026. (URL à confirmer avant publication)
- METR — *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*, juil. 2025. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/> (révision fév. 2026 — URL à confirmer)
- BairesDev — *Dev Barometer Q4 2025*. <https://www.bairesdev.com/blog/dev-barometer-q4-2025/>
- DX — *Core 4 / AI-assisted engineering Q4 impact report* (2026). Cadre de mesure unifiant DORA, SPACE et DevEx (Speed, Effectiveness, Quality, Business Impact). Gains d'efficacité d'ingénierie de 3 à 12 % sur plus de 300 organisations ; +14 % de temps redéployé vers la conception de fonctionnalités ; +15 % d'engagement. <https://getdx.com/report/ai-assisted-engineering-q4-impact-report/>
- Stanford HAI — *AI Index Report 2026*. <https://aiindex.stanford.edu/>
- Matt Garman (CEO d'AWS) — déclaration publique, 2025. (source primaire à confirmer)
- Arthur Mensch (CEO de Mistral AI) — Audition par la commission d'enquête sur les dépendances structurelles et les vulnérabilités systémiques du secteur numérique, Assemblée nationale, 12 mai 2026. <https://lcp.fr/actualites/intelligence-artificielle-le-patron-de-mistral-ai-arthur-mensch-n-exclut-pas-une-hausse> ; compte rendu complémentaire : <https://www.usine-digitale.fr/intelligence-artificielle/dans-lia-la-france-doit-agir-maintenant-pour-ne-pas-devenir-un-vassal-arthur-mensch-ceo-de-mistral-alerte-les-deputes-sur-lurgence-face-aux-geants-americains.X4GLZBM2DZGZZGVM6SPWQYQOZU.html>

12.1.2 Outils, frameworks et standards

- GitHub Spec Kit — <https://github.com/github/spec-kit>
- OpenSpec — <https://openspec.dev/>
- BMAD Method — <https://docs.bmad-method.org/>
- Zenika Zenflow — <https://github.com/zenika-open-source>
- Agentgateway — <https://agentgateway.dev/>
- Kong AI Gateway — <https://konghq.com/products/kong-ai-gateway>
- LiteLLM — <https://github.com/BerriAI/litellm>
- Red Hat OpenShift (AI / orchestration des workloads) — <https://www.redhat.com/en/technologies/cloud-computing/openshift>
- vLLM (moteur d'inférence haute performance) — <https://github.com/vllm-project/vllm>
- Qdrant — <https://qdrant.tech/>
- Weaviate — <https://weaviate.io/>
- OWASP AI Maturity Assessment (AIMA) — <https://owasp.org/www-project-ai-maturity-assessment/>

Auteurs



Olivier Coupelon

Head of AI Zenika FR



Adrien Nortain

CTO & Partner



Matthias Dugué

Head of Software
Engineering FR



Michael Isvy

Head of Software
Engineering SG



Norbert Jeff Nadir

AI Ops & Change
Management

En collaboration avec **WAVESTONE** sur les sujets transformation et organisation.



Raphaël Khalifa

Senior Manager, Head
of AI Powered PDLC
Transformation



Raphaël Brun

Partner, Digital Maker &
Product Design Expertise
Director

Pour en savoir plus

info@zenika.com

